

**TUGAS BESAR 2 IF3270 PEMBELAJARAN MESIN
SEMESTER II TAHUN 2025/2026**

**CONVOLUTIONAL NEURAL NETWORK DAN RECURRENT NEURAL
NETWORK**



Kelompok 42- KicauMania

Disusun oleh:

Muhammad Aufa Farabi	13523023
Abdullah Farhan	13523042
Ferdinand Gabe Tua Sinaga	13523051

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026**

Daftar Isi

BAGIAN I. DESKRIPSI PERSOALAN	3
BAGIAN II. PENJELASAN IMPLEMENTASI	4
BAGIAN III. PENJELASAN FORWARD PROPAGATION	9
BAGIAN IV. HASIL PENGUJIAN	11
BAGIAN IV. KESIMPULAN DAN SARAN	24
BAGIAN V. PEMBAGIAN TUGAS	25
BAGIAN VI. REFERENSI	25
SURAT PERNYATAAN PENGGUNAAN AI	26

BAGIAN I. DESKRIPSI PERSOALAN

Perkembangan deep learning dalam beberapa dekade terakhir telah melahirkan dua arsitektur fundamental yang menjadi tulang punggung berbagai sistem kecerdasan buatan modern yaitu Convolutional Neural Network (CNN) dan Recurrent Neural Network (RNN). CNN unggul dalam mengekstraksi fitur spasial dari data gambar, sementara RNN dan variannya Long Short-Term Memory (LSTM), dirancang untuk memodelkan dependensi sekuensial dalam data berurutan seperti teks.

Meskipun berbagai framework seperti TensorFlow dan Keras telah menyederhanakan proses pelatihan model, pemahaman mendalam terhadap mekanisme internal kedua arsitektur tersebut tetap krusial. Tanpa pemahaman tersebut, proses diagnosis kesalahan model, modifikasi arsitektur, maupun optimasi performa menjadi sulit dilakukan secara sistematis. Oleh karena itu, untuk meningkatkan pemahamannya tugas ini diimplementasikan. Secara garis besar tugas ini dapat dibagi menjadi 2 bagian besar.

Bagian pertama berfokus pada image classification menggunakan CNN dengan dataset Intel Image Classification yang terdiri dari sekitar 25.000 gambar dalam 6 kategori. Pada bagian ini diimplementasikan forward propagation CNN dari nol menggunakan NumPy, mencakup layer Conv2D dengan shared parameter, LocallyConnected2D dengan non-shared parameter, berbagai jenis pooling layer, serta fungsi aktivasi. Selain itu, dilakukan eksplorasi pengaruh hyperparameter seperti jumlah layer konvolusi, jumlah filter, ukuran filter, dan jenis pooling, serta perbandingan antara arsitektur shared dan non-shared parameter dari sisi akurasi dan efisiensi model.

Bagian kedua berfokus pada image captioning menggunakan arsitektur encoder-decoder berbasis CNN dan RNN/LSTM dengan dataset Flickr8k yang berisi 8.092 gambar beserta caption-nya. Arsitektur mengacu pada paper Show and Tell (Vinyals et al., 2015) dengan metode pre-inject, di mana fitur gambar dari CNN pretrained diproyeksikan sebagai input awal sebelum token <start>. Pada bagian ini diimplementasikan forward propagation SimpleRNN dan LSTM dari nol, kemudian dilakukan perbandingan menyeluruh antara keduanya dari sisi kualitas caption yang dihasilkan, skor BLEU-4 dan METEOR, serta waktu eksekusi.

BAGIAN II. PENJELASAN IMPLEMENTASI

Implementasi CNN

Implementasi	Keterangan
model.py (Class CNNModel)	Pada <code>model.py</code>, diimplementasikan kelas <code>CNNModel</code>, yaitu kelas untuk Model dari CNN. Berikut ini adalah atribut-atribut yang ada pada kelas <code>CNNModel</code>: - layers, list dari objek layer yang membentuk arsitektur CNN Beberapa metode yang terdapat pada kelas <code>CNNModel</code> adalah sebagai berikut: - add, menambahkan layer ke dalam model - load_weights_from_keras, memuat bobot dari semua layer Keras yang sudah terlatih ke layer scratch yang bersesuaian - forward, melakukan forward propagation melewati semua layer secara berurutan - predict, mengembalikan prediksi kelas dengan probabilitas tertinggi dari data input
conv2.py (Class Conv2D)	Layer konvolusi CNN direpresentasikan oleh Class <code>Conv2D</code> pada conv2.py. Berikut adalah atribut-atribut yang terdapat pada Kelas <code>Conv2D</code>: - stride, jumlah langkah pergeseran kernel saat sliding window - padding, metode padding yang digunakan, yaitu 'valid' atau 'same' - activation, objek fungsi aktivasi yang diterapkan setelah konvolusi (opsional) - kernel, tensor bobot kernel berukuran (kH, kW, C_in, C_out) - bias, vektor bias berukuran (C_out,) - inputs, variabel nilai input dari forward pass - inputs_padded, variabel input setelah proses padding - d_kernel, gradien dari fungsi loss terhadap kernel - d_bias, gradien dari fungsi loss terhadap bias Untuk metode-metode yang terdapat pada kelas <code>Conv2D</code>

	adalah seperti berikut: - load_weights, load bobot kernel dan bias dari layer Keras Conv2D yang sudah ditrain - forward, melakukan operasi konvolusi 2D dengan sliding window, dilanjutkan dengan fungsi aktivasi apabila ada - backward, menghitung gradien terhadap kernel, bias, dan input layer
locally_connected.py (LocallyConnected2D)	Layer locally connected pada CNN direpresentasikan oleh Kelas LocallyConnected2D pada locally_connected.py. Berbeda dengan layer Conv2D, layer ini menggunakan kernel yang berbeda untuk setiap posisi (non shared). Berikut adalah atribut-atribut yang ada pada kelas LocallyConnected2D: - stride, jumlah langkah pergeseran kernel saat sliding window - activation, objek fungsi aktivasi yang diterapkan setelah operasi (opsional) - kernel, tensor bobot dengan kernel yang berbeda untuk setiap posisi spasial - bias, vektor bias berukuran (H_out * W_out, C_out) - kernel_size, ukuran kernel berupa tuple (kH, kW) - inputs, variabel nilai input dari forward pass - d_kernel, gradien dari fungsi loss terhadap kernel - d_bias, gradien dari fungsi loss terhadap bias Berikut adalah metode-metode yang terdapat pada kelas LocallyConnected2D: - load_weights, metode untuk load bobot dan konfigurasi kernel dari layer Keras LocallyConnected2D yang sudah ditrain - forward, melakukan operasi locally connected 2D dengan sliding window menggunakan kernel yang unik per posisi spasial, dilanjutkan dengan fungsi aktivasi apabila ada - backward, menghitung gradien terhadap kernel, bias, dan input layer
pooling.py (Class MaxPooling2D, AveragePooling2D, GlobalAveragePooling2D)	File pooling.py berisi kelas-kelas yang mengimplementasikan operasi pooling pada CNN. Semua kelas pooling tersebut memiliki method-method yang sama, yaitu: - forward, menghitung output pooling untuk forward propagation

	- backward, menghitung gradien dan meneruskannya ke input layer - Adapun kelas-kelas yang terdapat pada pooling.py adalah sebagai berikut: 1. Kelas MaxPooling2D, kelas yang mengimplementasikan operasi Max Pooling 2D yang mengambil nilai maksimum dari setiap window. Atribut pada kelas ini adalah: - pool_size, size jendela pooling berupa tuple (pH, pW) - strides, jumlah langkah pergeseran jendela pooling berupa tuple (sH, sW) - inputs, variabel nilai input dari forward pass - is_batched, boolean apakah input berbentuk batch atau single 2. Kelas AveragePooling2D, kelas yang mengimplementasikan operasi Average Pooling 2D yang mengambil nilai rata-rata dari setiap window. Atribut pada kelas ini sama dengan MaxPooling2D. 3. Kelas GlobalMaxPooling2D, kelas yang mengimplementasikan operasi Global Max Pooling 2D yang mengambil nilai maksimum dari seluruh dimensi spasial setiap channel. Atribut pada kelas ini adalah: - inputs, menyimpan nilai input dari forward pass - is_batched, boolean apakah input berbentuk batch atau single 4. Kelas GlobalAveragePooling2D, mengimplementasikan operasi Global Average Pooling 2D yang mengambil nilai rata-rata dari seluruh dimensi spasial setiap channel. Atribut pada kelas ini sama dengan pada GlobalMaxPooling2D.
--	--

Implementasi RNN/LSTM

Implementasi RNN/LSTM dikerjakan untuk task image captioning pada dataset Flickr8k. Arsitektur yang digunakan mengikuti spesifikasi tugas, yaitu pre-inject: feature vector hasil CNN diproyeksikan terlebih dahulu oleh Image_Projection, lalu ditempatkan sebagai timestep $t=-1$ sebelum token `<start>`. Dengan demikian, sequence yang masuk ke decoder berbentuk `[image_context, emb(<start>), emb(S0), ..., emb(SN-1)]`, sedangkan targetnya adalah `[S0, S1, ..., SN]`.

Kontrak sequence yang digunakan adalah `sequence_length = 35`. Karena target caption digeser satu posisi, input teks decoder memiliki panjang 34 timestep (`seq[:-1]`) dan target juga memiliki panjang 34 timestep (`seq[1:]`). Output timestep dari image context tidak dipakai sebagai prediksi kata, sehingga pada model Keras ditambahkan `Drop_Image_Timestep` agar output model kembali sejajar dengan target teks.

Komponen	Implementasi	Keterangan
CNN encoder	<code>feature_extraction.py</code>	Mengekstraksi feature vector gambar Flickr8k menggunakan CNN pretrained tanpa classification head. Feature disimpan di <code>images_feature/</code> agar proses ekstraksi tidak perlu diulang saat training decoder.
Feature scaler	<code>experiment.py</code>	Feature gambar distandardisasi berdasarkan train split agar proyeksi fitur lebih stabil. Scaler disimpan sebagai <code>feature_scaler_preinject_v2.npz</code> .
Model Keras decoder	<code>modeling.py</code>	Membangun SimpleRNN/LSTM decoder dengan <code>Image_Input</code> , <code>Caption_Input</code> , <code>Image_Projection</code> , <code>Image_Timestep</code> , <code>PreInject_Concat</code> , layer recurrent, <code>Drop_Image_Timestep</code> , dan <code>Output_Layer</code> .
Data generator	<code>train_utils.py</code>	Menghasilkan pasangan input (<code>[X_img, X_txt]</code> , <code>y</code> , <code>sample_weight</code>). <code>X_txt</code> dan <code>y</code> sama-sama 34 timestep, sedangkan <code>sample_weight</code> digunakan agar token padding tidak ikut memengaruhi loss.
Training pipeline	<code>training.py</code>	Melatih 6 variasi SimpleRNN dan 6 variasi LSTM. Pipeline dapat melewati model yang weight-nya sudah tersedia, sehingga eksperimen tidak perlu mengulang training dari awal.
DenseProjectionLayer	<code>dense_projection.py</code>	Forward propagation dari Dense projection untuk memetakan feature 2048 dimensi menjadi context 256 dimensi. Bobot dan bias diambil dari layer Keras <code>Image_Projection</code> .
Embedding Layer	<code>embedding.py</code>	Melakukan lookup embedding token berdasarkan bobot <code>Token_Embedding</code> dari Keras.
RNNLayer	<code>rnn.py</code>	Implementasi SimpleRNN cell from scratch dengan rumus $h_t = \tanh(x_t W + h_{t-1} U + b)$.
LSTMLayer	<code>lstm.py</code>	Implementasi LSTM cell from scratch dengan input gate, forget gate, candidate cell, output gate, hidden state, dan cell state.
DenseOutputLayer	<code>dense_output.py</code>	Mengubah hidden state menjadi distribusi probabilitas vocabulary dengan Dense dan softmax.
Scratch caption model	<code>ImageCaptioningScratch.py</code>	Melakukan decoding dari bobot Keras memakai layer scratch. Pada awal decoding, model menjalankan satu <code>forward_step(image_context)</code> sebagai timestep $t=-1$, kemudian token <code><start></code> diproses pada timestep berikutnya.

Evaluasi	experiment.py	Menghitung BLEU-4, METEOR, waktu inferensi, jumlah caption unik, frekuensi caption paling sering, perbandingan Keras vs scratch, sweep panjang maksimum caption, dan qualitative samples.
----------	---------------	---

Variasi yang digunakan adalah Shallow_Small (1 layer, 128 hidden), Deep_Small (2 layer, 128 hidden), VeryDeep_Small (3 layer, 128 hidden), Shallow_Mid (1 layer, 256 hidden), Shallow_Large (1 layer, 512 hidden), dan Deep_Large (2 layer, 512 hidden). Keenam variasi tersebut dilatih untuk SimpleRNN dan LSTM sehingga total terdapat 12 model decoder.

BAGIAN III. PENJELASAN FORWARD PROPAGATION

Forward Propagation CNN

Mekanisme dari forward propagation pada model CNN yang diimplementasikan dilakukan pada metode forward dalam kelas CNNModel dengan setiap layer dari model melakukan metode forward yang dimilikinya secara berurutan. Alur dari *forward propagation* terdiri atas langkah-langkah seperti berikut:

1. Model CNN menerima data input berupa tensor gambar berformat (H, W, C) untuk single gambar atau (N, H, W, C) untuk batch gambar.
2. Setelah itu, Model melakukan pemanggilan forward untuk semua layer (termasuk aktivasi) dengan arah perhitungan maju dari layer pertama hingga terakhir secara berurutan.
3. Proses pada model CNN masuk ke dalam layer konvolusi (Conv2D). Apabila padding yang digunakan adalah 'same', input terlebih dahulu diberi padding nol di sekelilingnya agar ukuran output spasial nya terjaga. Selanjutnya, dilakukan operasi konvolusi di mana pada setiap posisi spasial (i, j), kernel K berukuran (kH, kW) digeser melalui input X menggunakan sliding window dengan langkah stride s uanh menghasilkan nilai output Z pada posisi tersebut melalui rumus $Z[i, j] = \sum X[i*s : i*s+kH, j*s : j*s+kW] \odot K + b$, di mana $\odot$ adalah perkalian element-wise yang diakumulasi. Proses tersebut feature map output berformat (N, H_out, W_out, C_out). Jika layer konvolusi memiliki fungsi aktivasi, fungsi aktivasi tersebut langsung diterapkan di dalam metode forward Conv2D itu sendiri sebelum output dikembalikan.
4. Output dari layer konvolusi diteruskan ke layer pooling yang melakukan downsampling pada dimensi spasial. MaxPooling2D mengambil nilai maksimum dari setiap window, $P[i, j] = \max(A[i*s : i*s+pH, j*s : j*s+pW])$, sedangkan AveragePooling2D mengambil nilai rata-ratanya, $P[i, j] = \text{mean}(A[i*s : i*s+pH, j*s : j*s+pW])$. Operasi ini mengurangi dimensi spasial (*downsampling*) sekaligus mempertahankan fitur yang paling dominan.
5. Langkah 3 dan 4 dapat diulangi sesuai dengan arsitektur model CNN yang dibangun, di mana mana setiap blok konvolusi-pooling menghasilkan representasi fitur yang semakin abstrak.
6. Output dari seluruh layer konvolusi dan pooling kemudian diratakan (*flattening*) oleh layer Flatten menjadi vektor satu dimensi per sampel berformat (N, H*W*C) untuk batch atau (H*W*C) untuk single, yang berfungsi sebagai jembatan antara bagian ekstraktor fitur spasial dengan bagian klasifikasi.
7. Setelah semua layer telah diproses, output dari layer terakhir menjadi hasil prediksi model (y_pred).

Forward Propagation RNN

Forward propagation pada SimpleRNN dimulai dari hidden state awal $h_0 = 0$. Pada arsitektur image captioning ini, timestep pertama bukan kata, melainkan image_context hasil proyeksi feature CNN. Timestep ini berperan sebagai x_{t-1} sesuai metode pre-inject. Setelah

timestep gambar diproses, hidden state yang terbentuk menjadi konteks awal untuk token <start> dan token-token caption berikutnya.

Untuk setiap timestep teks, embedding token x_t dikombinasikan dengan hidden state sebelumnya melalui operasi:

$$h_t = \tanh(x_t W + h_{t-1} U + b)$$
$$y_t = \text{softmax}(h_t W_o + b_o)$$

Output pada timestep image context dibuang karena target prediksi dimulai dari kata pertama caption. Oleh karena itu, model menghasilkan prediksi sepanjang 34 timestep, sama dengan panjang target seq[1:].

Forward Propagation LSTM

Forward propagation LSTM juga dimulai dengan $h_0 = 0$ dan $c_0 = 0$. Feature CNN yang sudah diproyeksikan masuk sebagai timestep $t = -1$, lalu memperbarui hidden state dan cell state sebelum token <start> diproses. Pada setiap timestep, LSTM menghitung empat komponen utama:

$$i_t = \text{sigmoid}(x_t W_i + h_{t-1} U_i + b_i)$$
$$f_t = \text{sigmoid}(x_t W_f + h_{t-1} U_f + b_f)$$
$$g_t = \tanh(x_t W_g + h_{t-1} U_g + b_g)$$
$$o_t = \text{sigmoid}(x_t W_o + h_{t-1} U_o + b_o)$$
$$c_t = f_t * c_{t-1} + i_t * g_t$$
$$h_t = o_t * \tanh(c_t)$$

Hidden state h_t kemudian diteruskan ke Dense output layer untuk menghasilkan distribusi probabilitas kata. Dibanding SimpleRNN, LSTM memiliki mekanisme gate yang memungkinkan model mengatur informasi mana yang disimpan atau dilupakan. Secara teori ini membantu mengurangi masalah vanishing gradient, tetapi pada eksperimen ini performa akhir tetap dipengaruhi oleh jumlah epoch, ukuran data, variasi arsitektur, dan kecocokan model terhadap konfigurasi pre-inject yang digunakan.

BAGIAN IV. HASIL PENGUJIAN

Hasil Pengujian CNN

Tag	Layers	Filters	Kernel	Pooling	F1

l2_f32_k3_max	2	32	3	max	0.7734
l1_f32_k3_max	1	32	3	max	0.7316
l2_f64_k5_max	2	64	5	max	0.6938
l2_f64_k3_max	2	64	3	max	0.6877
l1_f64_k3_avg	1	64	3	avg	0.6817
l2_f64_k3_avg	2	64	3	avg	0.6804
l1_f32_k3_avg	1	32	3	avg	0.6727
l1_f64_k3_max	1	64	3	max	0.6685
l2_f32_k3_avg	2	32	3	avg	0.6655
l1_f64_k5_avg	1	64	5	avg	0.6653
l2_f32_k5_max	2	32	5	max	0.6594
l1_f32_k5_max	1	32	5	max	0.6570
l2_f32_k5_avg	2	32	5	avg	0.6381
l1_f64_k5_max	1	64	5	max	0.6368
l1_f32_k5_avg	1	32	5	avg	0.6264
l2_f64_k5_avg	2	64	5	avg	0.5873

Gambar Hasil Ranking Model terbaik dari Training Model dengan 16 Variasi Hyperparameter

Model terbaik dari scratch diambil dengan variasi 2 convolution layer, 32 jumlah filters per layer, ukuran filter/kernel 3x3, dan max pooling (l2_f32_k3_max)

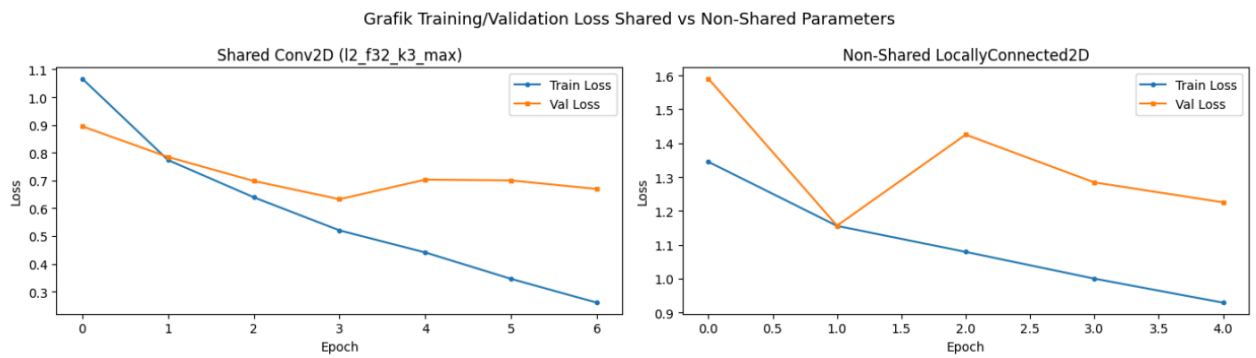
1. Perbandingan shared vs non-shared parameter

Pengujian pada bagian ini dilakukan dengan membandingkan model *scratch* yang memakai shared parameters dan non shared parameters (locally connected).

Model			Macro F1	Params
Keras	Shared	(Conv2D)	0.7734	1,059,622
Scratch	Shared	(Conv2D)	0.7734	1,059,622
Keras	Non-Shared	(LC2D)	0.5444	12,025,510
Scratch	Non-Shared	(LC2D)	0.5444	12,025,510

Rasio parameter Non-Shared vs Shared: 11.3x

Gambar Perbandingan macro F1-score, jumlah parameter shared vs non-shared



Gambar Perbandingan Train Loss dan Val Loss Shared vs Non-Shared Parameters CNN

Model shared (dari layer Conv2D) menghasilkan macro F1-score sebesar 0.7734 dengan jumlah parameter 1.059.622, sedangkan model non-shared (LocallyConnected2D) menghasilkan macro F1-score 0.5444 dengan jumlah parameter 12.025.510, yakni memiliki rasio 11.3x lebih banyak. Meskipun non-shared memiliki kapasitas parameter yang jauh lebih besar, performanya justru lebih rendah dengan selisih skor sebesar 0.2290. Hal ini disebabkan oleh tidak adanya parameter sharing sehingga model tidak dapat belajar fitur yang bersifat translation-invariant. Pada klasifikasi gambar, fitur seperti tepi, tekstur, dan pola objek dapat muncul di berbagai posisi spasial, sehingga shared parameter yang memaksa kernel yang sama diaplikasikan di seluruh posisi justru membantu generalisasi model.

Dari grafik training dan validation loss, model shared (l2_f32_k3_max) menunjukkan penurunan loss yang konsisten dan smooth di mana train loss turun dari sekitar 1.0 ke 0.3 dan val loss mengikuti dengan gap yang kecil. Di sini model dikatakan berhasil belajar tanpa overfitting signifikan. Sebaliknya, model non-shared menunjukkan kurva yang lebih tidak stabil dengan val loss yang naik turun, mengindikasikan model kesulitan menggeneralisasi meskipun train loss-nya menurun.

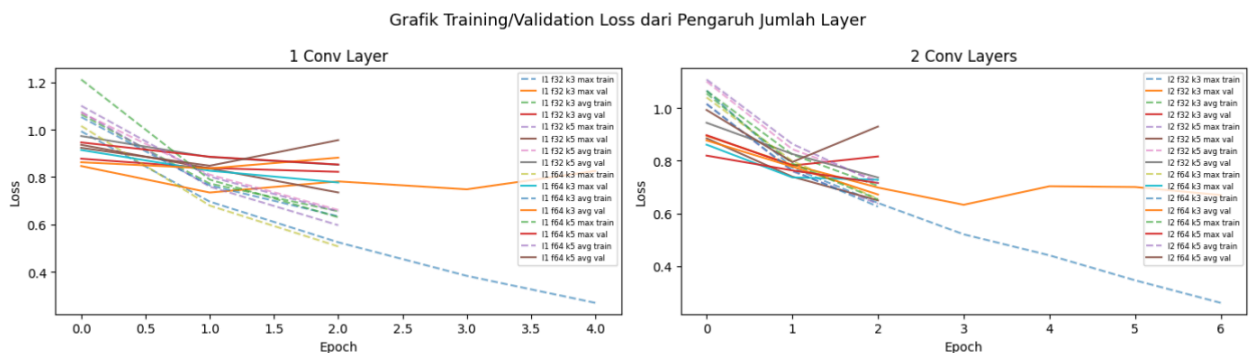
2. Pengaruh jumlah layer konvolusi

Pengujian pada bagian ini dilakukan dengan membandingkan hasil akhir prediksi dan grafik training/validation loss dari dua variasi parameter jumlah layer konvolusi

Perbandingan F1-Score per Pengaruh Jumlah Layer

Variasi	Avg F1	Max F1	Min F1
1 Convolution Layer	0.6675	0.7316	0.6264
2 Convolution Layers	0.6732	0.7734	0.5873

Gambar Perbandingan F1-Score Berdasarkan Variasi Pengaruh Jumlah Layer



Gambar Perbandingan Train Loss dan Val Loss dari Variasi Jumlah Layer Konvolusi

Berdasarkan tabel perbandingan F1-score yang berkaitan dengan hasil akhir prediksi variasi model ini, model dengan 2 convolution layer menghasilkan rata-rata F1 lebih tinggi, dengan avg 0.6732, max 0.7734). dibanding model dengan 1 convolution layer, dengan avg 0.6675, max 0.7316). Hal ini sesuai dengan teori bahwa penambahan layer konvolusi memungkinkan model membangun representasi fitur yang lebih hierarkis di mana layer pertama menangkap fitur low-level seperti tepi dan warna, sedangkan layer kedua menggabungkannya menjadi fitur yang lebih kompleks seperti tekstur dan bentuk objek.

Dari grafik training dan validation loss, model dengan 2 convolution layer umumnya mencapai nilai loss yang lebih rendah pada akhir training. Hal tersebut terlihat dari beberapa garis yang turun lebih ke bawah pada panel kanan. Perbedaan panjang garis antar model pada grafik disebabkan oleh adanya early stopping yang menghentikan training saat validation loss tidak membaik selama 3 epoch berturut-turut. Walaupun begitu, perbedaan rata-rata F1 antara 1 dan 2 layer relatif kecil (0.0057). Hal ini mengindikasikan bahwa untuk dataset dan ukuran gambar 64×64 , 1 layer konvolusi sudah cukup menangkap fitur diskriminatif meskipun 2 layer memberikan sedikit keunggulan.

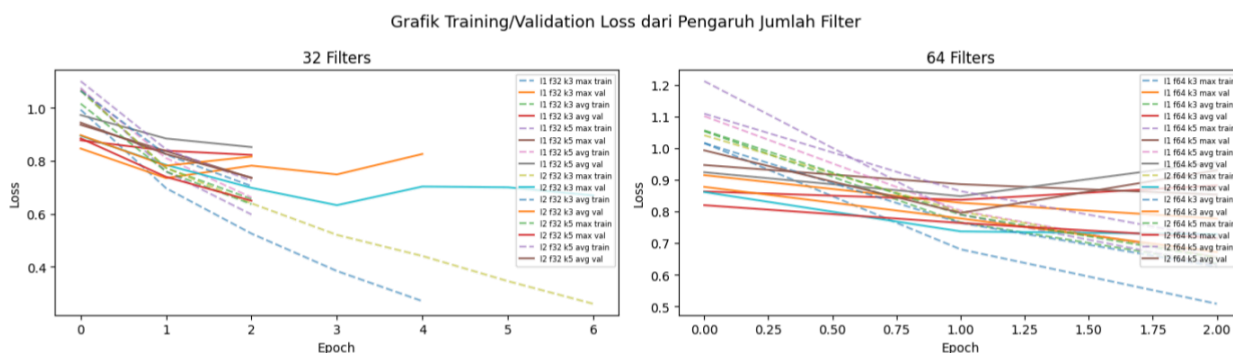
3. Pengaruh banyak filter per layer konvolusi

Pengujian pada bagian ini dilakukan dengan membandingkan hasil akhir prediksi dan grafik training/validation loss dari dua variasi parameter ukuran filter per layer konvolusi:

Perbandingan F1-Score per Pengaruh Jumlah Filter

Variasi	Avg F1	Max F1	Min F1
32 Filters	0.6780	0.7734	0.6264
64 Filters	0.6627	0.6938	0.5873

Gambar Perbandingan F1-Score Berdasarkan Variasi Banyak Filter per Layer Konvolusi

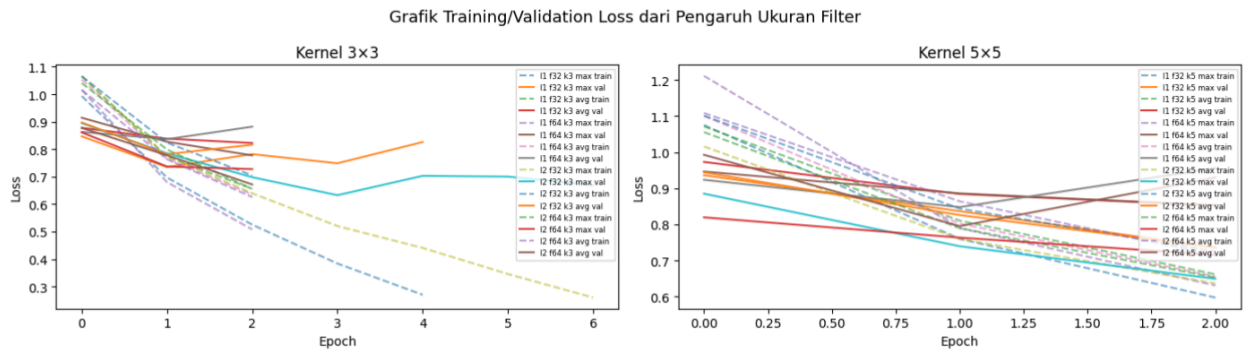


Gambar Perbandingan Train Loss dan Val Loss dari Variasi Banyak Filter per Layer Konvolusi

Hasil eksperimen terkait variasi banyak filter per konvolusi menunjukkan bahwa model dengan 32 filter menghasilkan rata-rata skor F1 yang lebih tinggi, dengan avg 0.6780, max 0.7734, dibandingkan model dengan 64 filter, dengan avg 0.6627, max 0.6938. Hal ini cukup berlawanan dengan asumsi umum bahwa semakin banyak filter dapat menghasilkan representasi yang lebih kaya. Hasil seperti tersebut dapat dijelaskan oleh ukuran dataset yang 14.000 gambar training dan kompleksitas gambar berukuran 64×64 sehingga model dengan 64 filter memiliki terlalu banyak parameter sehingga lebih mudah overfit. Dari grafik training dan validation loss, model dengan 64 filter menunjukkan training epoch yang lebih singkat (sekitar 2 epoch) dibanding 32 filter (5-6 epoch), mengindikasikan early stopping berhenti lebih cepat karena val loss tidak membaik. Hal ini konsisten dengan tanda overfitting ringan. Model dengan 32 filter sudah cukup untuk menangkap fitur-fitur diskriminatif dari 6 kategori pada dataset ini dengan generalisasi yang lebih baik.

4. Pengaruh ukuran filter per layer konvolusi

Pengujian pada bagian ini dilakukan dengan membandingkan hasil akhir prediksi dan grafik training/validation loss dari dua variasi parameter pooling layer



Gambar Perbandingan F1-Score Berdasarkan Variasi Ukuran Filter per Layer Konvolusi

Perbandingan F1-Score per Pengaruh Ukuran Filter			
Variasi	Avg F1	Max F1	Min F1
Kernel 3x3	0.6952	0.7734	0.6655
Kernel 5x5	0.6455	0.6938	0.5873

Gambar Perbandingan Train Loss dan Val Loss dari Variasi Ukuran Filter per Layer Konvolusi

Hasil pengujian terhadap variasi ukuran filter per layer konvolusi menunjukkan bahwa model dengan kernel 3×3 secara rata-rata menghasilkan F1-score, yang terkait dengan hasil prediksi model, lebih tinggi, avg F1 0.6952, max F1 0.7734, dibanding kernel 5×5, avg F1 0.6455, max F1 0.6938, dengan selisih avg F1 sebesar 0.0497 yang cukup signifikan. Kernel 3×3 lebih efisien karena memiliki parameter yang lebih sedikit per filter tetapi tetap dapat menangkap fitur lokal dengan baik. Pada gambar berukuran 64×64, *receptive field* kernel 3×3 sudah cukup untuk mendeteksi fitur seperti tepi dan tekstur yang relevan untuk membedakan kategori buildings, forest, glacier, mountain, sea, dan street. Kernel 5×5 memiliki *receptive field* lebih besar yang secara teori dapat menangkap konteks spasial lebih luas, tetapi yang terjadi untuk ukuran gambar dengan jumlah dataset yang diuji tidak sesuai dengan kasus umum tersebut, yang justru malah menambah parameter yang tidak produktif. Dari grafik training/validation loss, model kernel 3×3 umumnya menunjukkan training yang lebih panjang dan konvergensi lebih stabil dibanding kernel 5×5 yang cenderung berhenti lebih awal.

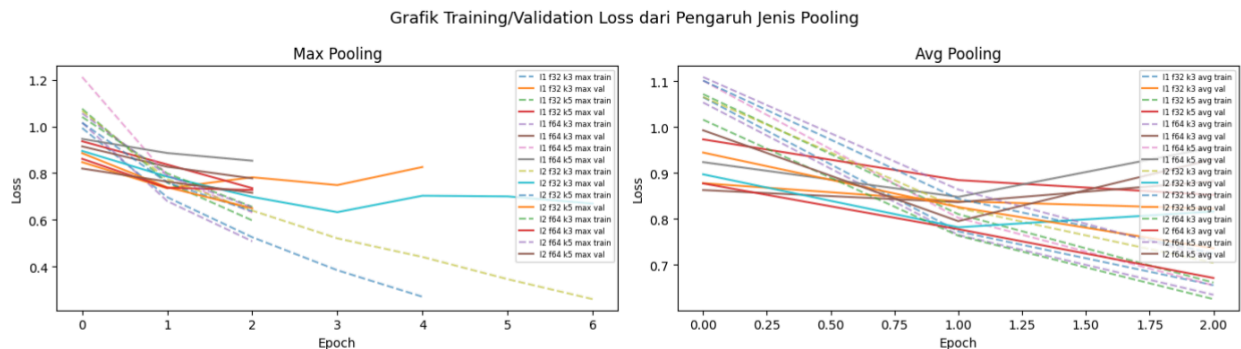
5. Pengaruh jenis pooling layer

Pengujian pada bagian ini dilakukan dengan membandingkan hasil akhir prediksi dan grafik training/validation loss dari dua variasi parameter pooling layer

Perbandingan F1-Score per Pengaruh Jenis Pooling

Variasi	Avg F1	Max F1	Min F1
Max Pooling	0.6885	0.7734	0.6368
Avg Pooling	0.6522	0.6817	0.5873

Gambar Perbandingan F1-Score Berdasarkan Variasi Pengaruh Jenis Pooling Layer



Gambar Perbandingan Train Loss dan Val Loss dari Variasi Jenis Pooling Layer

Berdasarkan hasil pengujian variasi jenis pooling layer, jenis Max pooling secara konsisten menghasilkan F1-score yang lebih tinggi, yakni avg F1 0.6885, max F1 0.7734. dibandingkan average pooling, yaitu avg F1 0.6522, max F1 0.6817 dengan selisih avg F1 nya adalah 0.0363. Hal ini sesuai dengan karakteristik kedua jenis pooling di mana max pooling mengambil nilai maksimum dari setiap window sehingga lebih baik dalam mempertahankan fitur paling menonjol seperti keberadaan tepi tajam atau tekstur kuat, sedangkan average pooling merata-ratakan semua nilai sehingga dapat meredam sinyal fitur yang penting. Untuk task klasifikasi gambar alam seperti dari dataset gambar yang diuji disini yang mana keberadaan fitur tertentu (misalnya warna biru dominan untuk sea, atau tekstur putih untuk glacier) lebih penting daripada distribusi rata-ratanya, max pooling terbukti lebih efektif.

Dari hasil grafik training and validation loss, model dengan max pooling umumnya mencapai nilai validation loss yang lebih rendah dibanding average pooling pada konfigurasi hyperparameter yang sama, yang terlihat dari beberapa *line* yang turun lebih dalam pada sisi kiri.

6. Perbandingan Keras vs from scratch

Pengujian pada bagian ini dilakukan dengan membandingkan hasil Forward Propagation dari model Scratch dengan model Keras

F1 Keras (shared): 0.7734 | Waktu: 1.59s

F1 Scratch (shared): 0.7734 | Waktu: 23.24s

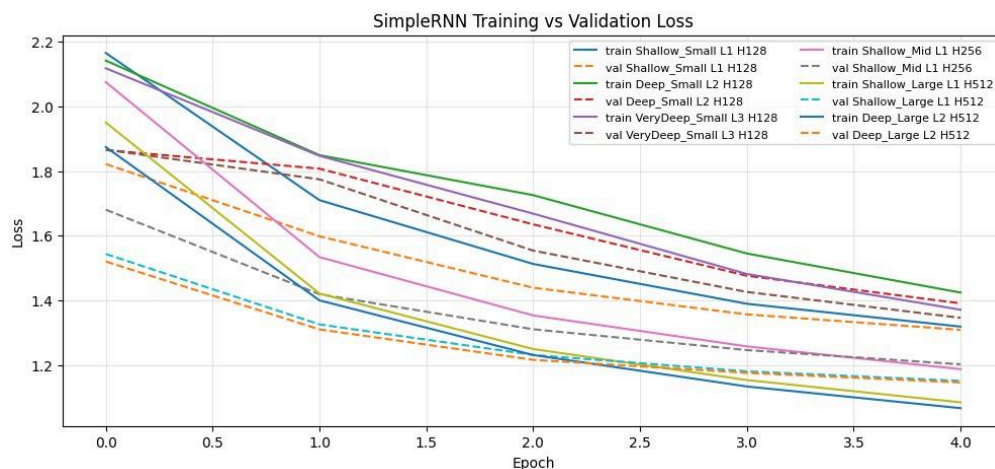
Gambar Perbandingan Hasil Macro-F1 Score Forward Pass Keras vs Scratch (Shared)

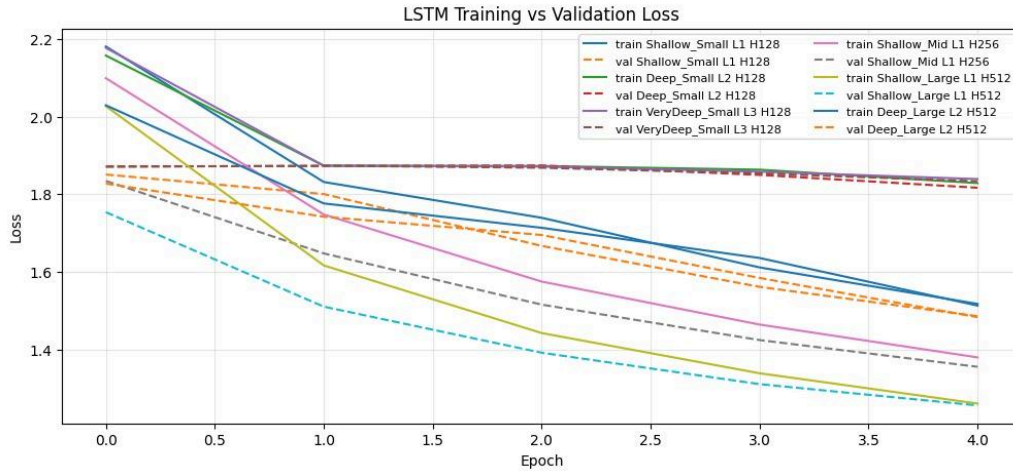
Berdasarkan gambar hasil pengujian di atas, Forward propagation dari model scratch menghasilkan F1-score yang identik dengan model Keras, yakni 0.7734, yang menunjukkan bahwa implementasi forward propagation secara *scratch* sudah benar dan konsisten dengan pemakaian library Keras. Namun, terdapat perbedaan utama terletak pada waktu eksekusi di mana dengan library Keras hanya membutuhkan 1.59 detik dalam proses, sedangkan scratch membutuhkan 23.24 detik untuk seluruh test set, yang jika dibuat rasio sekitar 14.6x lebih lambat. Perbedaan ini disebabkan oleh beberapa faktor, seperti Keras yang memanfaatkan akselerasi GPU melalui CUDA, operasi konvolusi dari Keras sudah dioptimasi di dengan C++, serta Keras mendukung komputasi paralel antar batch yang hal-hal tersebut tidak dilakukan secara implementasi *scratch* (memakai CPU dan tidak paralel).

Hasil Pengujian RNN/LSTM

Eksperimen RNN/LSTM menggunakan artefak preinject_v2. Semua hasil di bawah berasal dari output notebook dan CSV pada artifacts/rnn/results/. Evaluasi dilakukan pada split test Flickr8k dengan metrik BLEU-4, METEOR, waktu inferensi rata-rata per gambar, jumlah caption unik, dan frekuensi caption paling sering. Jumlah caption unik dan frekuensi caption paling digunakan sebagai indikator tambahan untuk mendeteksi apakah model mengalami caption collapse.

1. Variasi Jumlah Layer dan Hidden State





Gambar Perbandingan Train Loss dan Val Loss RNN dan LSTM

Tabel Perbandingan untuk setiap variasi jumlah layer dan ukuran hidden

Model	Nama Variasi	Layer	Hidden State	Scratch			Unique Caps	Top Cap Freq	Final Loss	Final Val Loss
				Bleu-4	Meteor	Avg Time				
SimpleR NN	Shallow Small	1	128	0.0920	0.2727	0.1347	3	70%	1.3190	1.3090
SimpleR NN	Deep Small	2	128	0.0878	0.2735	0.0768	3	63%	1.4242	1.3908
SimpleR NN	Very Deep Small	3	128	0.0862	0.2529	0.0773	13	25%	1.3711	1.3460
SimpleR NN	Shallow Mid	1	256	0.1022	0.3046	0.1230	38	22%	1.1874	1.2024
SimpleR NN	Shallow Large	1	512	0.1325	0.3305	0.3154	88	6%	1.0849	1.1511
SimpleR NN	Deep Large	2	512	0.1123	0.3339	0.6741	80	9%	1.0669	1.1463
LSTM	Shallow Small	1	128	0.0632	0.2339	0.2165	10	36%	1.5175	1.4858
LSTM	Deep Small	2	128	0.0102	0.0946	0.3185	6	44%	1.8289	1.8169
LSTM	Very Deep Small	3	128	0.0354	0.1111	0.1357	3	80%	1.8393	1.8336
LSTM	Shallow Mid	1	256	0.0648	0.2637	0.4976	15	35%	1.3796	1.3555
LSTM	Shallow Large	1	512	0.0990	0.3089	0.6341	39	8%	1.2607	1.2559
LSTM	Deep Large	2	512	0.0512	0.2072	1.4567	65	23%	1.5132	1.4834

Berdasarkan grafik training loss dan validation loss di atas, secara umum kedua arsitektur rekuren (SimpleRNN dan LSTM) menunjukkan kemampuan untuk mempelajari pola data sekuensial yang ditandai dengan tren penurunan nilai loss pada mayoritas variasi model. Meskipun demikian, jika diperhatikan lebih lanjut, beberapa variasi pada model LSTM menunjukkan gejala stagnasi. Hal ini terlihat dari kurva training loss yang cenderung mendatar setelah epoch 2 dan nilai validation loss yang konstan hingga akhir epoch 5, menandakan model mengalami kesulitan dalam mengoptimasi bobotnya pada fase awal pelatihan.

Berdasarkan BLEU-4, model terbaik untuk SimpleRNN adalah Shallow_Large dengan hidden state 512, BLEU-4 0.1325, METEOR 0.3305, 88 caption unik, dan top-caption frequency 6%. Model terbaik untuk LSTM adalah Shallow_Large dengan hidden state 512, BLEU-4 0.0990, METEOR 0.3089, 39 caption unik, dan top-caption frequency 8%.

Pengaruh hidden state terlihat jelas. Pada SimpleRNN, kenaikan hidden state dari 128 ke 256 dan 512 meningkatkan BLEU-4 dari sekitar 0.09 menjadi 0.1022 dan 0.1325. Jumlah caption unik juga meningkat dari 3 caption pada variasi 128 hidden menjadi 88 caption pada Shallow_Large. Ini menunjukkan bahwa kapasitas representasi yang lebih besar membantu model menangkap variasi visual dan bahasa dengan lebih baik. Pada LSTM, pola serupa terlihat pada variasi shallow, Shallow_Small menghasilkan BLEU-4 0.0632, Shallow_Mid 0.0648, dan Shallow_Large 0.0990.

Penambahan layer tidak selalu meningkatkan performa. Pada SimpleRNN hidden 128, model 1 layer, 2 layer, dan 3 layer menghasilkan BLEU-4 0.0920, 0.0878, dan 0.0862. Pada LSTM hidden 128, model 2 dan 3 layer justru turun tajam menjadi 0.0102 dan 0.0354. Hal ini menunjukkan bahwa model yang lebih dalam tidak otomatis lebih baik ketika epoch terbatas dan dataset relatif kecil. Model deep juga membutuhkan waktu inferensi lebih besar, misalnya LSTM Deep_Large mencapai 1.4567 detik per gambar.

2. Perbandingan Keras vs Scratch

Tabel Perbandingan keras vs scratch

Model	Variasi	Implement	Bleu-4	Meteor	Avg Time	Unique Cap	Top Cap Frequency
LSTM	Shallow Large	Scratch	0.0990	0.3089	0.6622	39	8%
LSTM	Shallow Large	Keras	0.0990	0.3089	4.3979	39	8%
RNN	Shallow Large	Scratch	0.1325	0.3305	0.5762	88	6%
RNN	Shallow	Keras	0.1325	0.3305	2.7524	88	6%

	Large						
--	-------	--	--	--	--	--	--

Hasil Keras dan scratch memiliki BLEU-4, METEOR, jumlah caption unik, dan top-caption frequency yang sama. Ini menunjukkan bahwa implementasi forward propagation scratch sudah konsisten dengan model Keras untuk arsitektur pre-inject. Perbedaan utama terdapat pada waktu inferensi. Scratch lebih cepat pada eksperimen ini karena proses greedy decoding dilakukan langsung dengan NumPy dan tidak memanggil model.predict Keras berulang kali pada setiap timestep. Untuk SimpleRNN Shallow_Large, scratch membutuhkan 0.5762 detik per gambar, sedangkan Keras 2.7524 detik per gambar. Untuk LSTM Shallow_Large, scratch membutuhkan 0.6622 detik per gambar, sedangkan Keras 4.3979 detik per gambar.

3. Perbandingan RNN vs LSTM

Model terbaik keseluruhan berdasarkan BLEU-4 adalah SimpleRNN Shallow_Large dengan BLEU-4 0.1325 dan METEOR 0.3305. Pada konfigurasi eksperimen ini, SimpleRNN Shallow_Large mengungguli LSTM Shallow_Large dari sisi BLEU-4, METEOR, waktu inferensi, dan keragaman caption. SimpleRNN menghasilkan 88 caption unik dengan top-caption frequency 6%, sedangkan LSTM menghasilkan 39 caption unik dengan top-caption frequency 8%.

Secara teori LSTM lebih kuat untuk dependensi jangka panjang karena memiliki cell state dan gate. Namun, hasil eksperimen menunjukkan bahwa untuk setup ini SimpleRNN lebih efektif. Penyebab utamanya adalah caption Flickr8k relatif pendek, training dilakukan 5 epoch, dan model terbaik yang muncul adalah konfigurasi shallow dengan hidden state besar. Pada kondisi tersebut, kapasitas hidden state 512 pada SimpleRNN cukup untuk menangkap pola caption umum tanpa overhead gate LSTM. LSTM tetap menghasilkan caption yang masuk akal pada beberapa contoh, tetapi beberapa variasi LSTM yang lebih dalam menunjukkan performa rendah dan tanda repetisi kalimat.

Tabel qualitative analysis

Image Name	Generated Caption LSTM	Generated Caption SimpleRNN	Bleu-4 LSTM	Bleu-4 Simple RNN	Ground Truth Ringkas
	a dog is running through the snow	a black dog is running through the grass	0.7290	0.8409	a big black dog bares its teeth while running through the snow

	a man in a red shirt and a black and white and white and white and white and white and white and white and white and white and white and white and	a man in a red shirt and a woman in a red shirt and a black and white dog	0.0146	0.0283	four people are lining up to purchase tickets at the theater
	a dog is running through the water	a man is jumping over a hurdle	0.0720	0.0670	a green fish is jumping out of water
	a young boy is jumping on a rock	a girl in a blue shirt is jumping on a swing	0.0684	0.0718	one little girl dressed in pink watches another also wearing some pink jump on the trampoline

Pada gambar anjing di salju dan rumput, kedua model mampu menghasilkan caption bertema anjing dan aktivitas berlari. Contoh terbaik adalah 2335619125_2e2034f2c3.jpg, dengan BLEU-4 SimpleRNN 0.8409 dan LSTM 0.7290. Pada kasus yang lebih sulit, seperti gambar prosesi pemakaman, antrian tiket, ikan melompat, atau anak di trampolin, model cenderung jatuh ke pola caption umum seperti "a man in a red shirt" atau "a dog is running". Ini menunjukkan bahwa model masih kuat pada objek/aksi yang sering muncul di data, tetapi belum cukup baik untuk objek atau konteks visual yang jarang.

4. Pengaruh panjang maksimum caption

Tabel pengaruh panjang maksimum caption

Max Len	Model Type	Variation Name	Implementation	Bleu-4	Meteor	Avg Time sec	Unique Captions	Top Caption Frequency
10	Simple RNN	Shallow_Large	scratch	0.1392	0.3265	0.4585	77	6%

20	Simple RNN	Shallow_Large	scratch	0.1326	0.3305	0.5698	88	6%
35	Simple RNN	Shallow_Large	scratch	0.1325	0.3305	0.4721	88	6%

Variasi panjang maksimum caption dilakukan pada model terbaik, yaitu SimpleRNN Shallow_Large scratch. Panjang maksimum 10 menghasilkan BLEU-4 tertinggi, yaitu 0.1392, sementara panjang 20 dan 35 menghasilkan BLEU-4 sekitar 0.1325. METEOR sedikit lebih tinggi pada panjang 20/35, tetapi BLEU-4 lebih sensitif terhadap frasa pendek yang tepat. Dengan panjang maksimum terlalu besar, model memiliki peluang menghasilkan kata tambahan atau repetisi yang menurunkan presisi n-gram. Karena itu, untuk setup ini panjang maksimum 10 paling baik berdasarkan BLEU-4, sedangkan panjang 20/35 masih berguna jika ingin caption yang lebih lengkap.

BAGIAN IV. KESIMPULAN DAN SARAN

Pada bagian CNN, implementasi forward propagation dari model *scratch* dengan memakai NumPy berhasil dibangun dan divalidasi yang menghasilkan prediksi yang identik dengan model Keras pada seluruh konfigurasi yang diuji, dengan macro F1-score 0.7734 untuk model terbaik dari semua variasi hyperparameter. Dari 16 variasi hyperparameter, yaitu 2 variasi jumlah layer konvolusi, ukuran filter per layer konvolusi, dan jenis pooling layer yang diuji untuk melatih model, model terbaik adalah konfigurasi dengan 2 convolution layer, 32 filter, kernel 3×3 , dan max pooling. Hasil pengujian menunjukkan bahwa 2 convolution layer umumnya lebih baik dari 1 layer, kernel 3×3 lebih unggul dari 5×5 , ukuran 32 filter lebih optimal dari 64 filter untuk dataset Intel Image Classification yang berukuran sedang ini, dan max pooling secara konsisten mengungguli average pooling. Perbandingan shared versus non-shared parameter untuk model *scratch* menunjukkan bahwa model dengan parameter sharing (memakai layer Conv2D) menghasilkan performa yang jauh lebih baik (F1 Score 0.7734 vs 0.5444) sekaligus lebih efisien dari sisi jumlah parameter (1.05 juta vs 12 juta) yang tidak hanya membuat model lebih efisien secara komputasi dan memori, tetapi juga meningkatkan kemampuan generalisasi secara signifikan untuk task klasifikasi gambar.

Pada bagian RNN/LSTM, sistem image captioning berhasil dibangun menggunakan arsitektur pre-inject, yaitu feature gambar dari CNN pretrained diproyeksikan menjadi image context lalu dimasukkan sebagai timestep awal sebelum token <start>. Decoder SimpleRNN dan LSTM dilatih pada Flickr8k dengan teacher forcing, kemudian bobot hasil training digunakan juga pada implementasi forward propagation from scratch. Hasil Keras dan scratch konsisten, dengan nilai BLEU-4 dan METEOR yang sama pada model terbaik, sehingga implementasi scratch dapat dikatakan sudah mengikuti perilaku model Keras.

Dari 12 variasi model, hasil terbaik diperoleh oleh SimpleRNN Shallow_Large dengan 1 layer dan hidden state 512, menghasilkan BLEU-4 0.1325 dan METEOR 0.3305. Peningkatan hidden state terbukti membantu kualitas dan keragaman caption, sedangkan penambahan layer tidak selalu meningkatkan performa dan cenderung menambah waktu inferensi. Pada eksperimen ini, SimpleRNN mengungguli LSTM karena caption Flickr8k relatif pendek dan konfigurasi shallow dengan hidden state besar sudah cukup efektif. Secara kualitatif, model cukup baik pada pola visual umum seperti anjing berlari atau orang melakukan aktivitas luar ruangan, tetapi masih cenderung menghasilkan caption generik pada gambar dengan konteks yang lebih spesifik.

BAGIAN V. PEMBAGIAN TUGAS

NIM	Nama	Tugas
13523023	Muhammad Aufa Farabi	Implementasi CNN
13523042	Abdullah Farhan	Implementasi RNN LSTM (Bagian 0 dan 4 sampai 6), Perbaikan Arsitektur
13523051	Ferdinand Gabe Tua Sinaga	Implementasi RNN LSTM (Bagian 1 sampai 3)

BAGIAN VI. REFERENSI

- [d2l.ai: LSTM](#)
- [d2l.ai: Deep RNN](#)
- [d2l.ai: Bidirectional RNN](#)
- [d2l.ai: RNN chapter](#)
- [d2l.ai: CNN chapter](#)
- [d2l.ai: Image Captioning](#)

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM :

1. Muhammad Aufa Farabi / 13523023
2. Abudullah Farhan / 13523042
3. Ferdinand Gabe Tua Sinaga / 13523051

Fakultas/Sekolah : STEI

Kode Mata Kuliah : IF3270

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas/Proposal : Tugas Besar 2 IF3270 Pembelajaran Mesin

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah: Gemini

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Ringan

Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Ringan
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung 16 Maret 2026



Muhammad Aufa Farabi, 13523023

Bandung 16 Maret 2026



Abdullah Farhan, 13523042

Bandung 16 Maret 2026



Ferdinand Gabe Tua Sinaga,
13523051